

category: avalonia

tags: [migration, winforms, kztekkomponentavalonia, kz-component, wrapper, metadatacontext, workaround, reflection]

severity: high

created: 2026-07-14

updated: 2026-07-14

project-origin: Kztek.Camera migrate WinForms→Avalonia — G3.6 KzZoneEditor verify + G5.1 RegionsEditorControl wrap

Wrap Kz Avalonia component + 3-workaround pattern (khi thiếu API vs WinForms control gốc)

Tình huống gặp phải

Khi migrate WinForms → Avalonia trong hệ sinh thái KZTEK, thay vì build mới control Avalonia thuần, có thể ưu tiên **wrap component có sẵn** trong `KztekComponentAvalonia.dll` (pre-built, không có source trong repo). Nhưng Kz Avalonia component thường có **API surface ĐƠN GIẢN hơn** WinForms control gốc (tối ưu cho design use case chung, không cover mọi edge case).

Ví dụ cụ thể — port `DefineRegionsControl` (WinForms, 315 dòng, có `BackgroundImage/DrawingMode/Rectangles[]/scale image↔canvas`) → `KzZoneEditor` (Kz Avalonia, có `Zones/ZoneAdded event/ClearZones()` nhưng KHÔNG có `BackgroundImage`, KHÔNG có mode toggle, KHÔNG có auto-scale).

Câu hỏi cốt lõi: wrap Kz component hay build mới control Avalonia?

Nguyên nhân gốc rễ (Root Cause)

- `KztekComponentAvalonia.dll` pre-built (không có source công khai trong repo) → không thể chỉnh sửa trực tiếp control để bổ sung tính năng thiếu.
- Kz Avalonia control ưu tiên simplicity + brand consistency → intentionally không cover edge case.
- Build mới control Avalonia thuần (thay Kz) tốn nhiều effort (5-10× thời gian wrap), và phải maintain thêm 1 codebase custom control.

Giải pháp — Verify API + 3-workaround wrap

Bước 1 — Verify API bằng MetadataLoadContext (reflection-only, không cần source)

Trước khi quyết định wrap hay build mới, **PHẢI liệt kê API surface thực của Kz component** — không dựa vào XML docs (thường thiếu member) hay tên control (dễ gây kỳ vọng sai).

Tạo scratch script `_workspace/kzprobe/`:

```
using System.Reflection;
using System.Runtime.InteropServices;
```

```
// Runtime + Avalonia deps (khớp version)
var runtime = Directory.GetFiles(RuntimeEnvironment.GetRuntimeDirectory(), "*.dll");
var avalonia = Directory.GetFiles(
    Path.Combine(NugetCache, "avalonia", "11.2.7", "lib", "net8.0"), "*.dll");
var paths = new HashSet<string>(runtime).Concat(avalonia).Append(kzDllPath).ToArray();

var resolver = new PathAssemblyResolver(paths);
using var mlc = new MetadataLoadContext(resolver);
var asm = mlc.LoadFromAssemblyPath(kzDllPath);
var t = asm.GetType("KztekComponentAvalonia.Controls.KzZoneEditor");

// List public API
foreach (var p in t.GetProperties(BindingFlags.Public | BindingFlags.Instance |
    BindingFlags.DeclaredOnly))
    Console.WriteLine($"{p.PropertyType.Name} {p.Name} {{ { (p.CanRead?"get":"" ) }
    { (p.CanWrite?"set":"" ) } }}");
foreach (var e in t.GetEvents(BindingFlags.Public | BindingFlags.Instance |
    BindingFlags.DeclaredOnly))
    Console.WriteLine($"event {e.EventHandlerType?.Name} {e.Name}");
foreach (var m in t.GetMethods(BindingFlags.Public | BindingFlags.Instance |
    BindingFlags.DeclaredOnly).Where(m => !m.IsSpecialName))
    Console.WriteLine($"{m.ReturnType.Name} {m.Name} ({string.Join(", ",
    m.GetParameters().Select(x => $"{x.ParameterType.Name} {x.Name}"))}");
```

`MetadataLoadContext` reflect-only KHÔNG cần Avalonia runtime hoạt động → an toàn chạy trong console app tối giản. Nhớ thêm `PackageReference System.Reflection.MetadataLoadContext 8.0.0`.

Bước 2 — Tiêu chí quyết định: kết luận A/B/C

Liệt kê 5-6 câu hỏi cụ thể cho tính năng WinForms control gốc, đánh giá match với Kz Avalonia API vừa probe được:

Kết luận	Điều kiện	Hành động
---	---	---
A — Phù hợp	≥ 5/6 tính năng match trực tiếp	Dùng Kz component thẳng, không wrap
B — Cần build mới	Thiếu ≥ 2/6 tính năng CRITICAL, không có workaround khả thi	Build custom control Avalonia (Render override)
C — Phù hợp một phần	Thiếu 1-3 tính năng nhưng workaround khả thi thuần XAML+VM	Wrap + 3-workaround (§Bước 3) — 80% tiết kiệm effort so B

Thực tế project Kztek.Camera: `KzZoneEditor` → 3/6 Yes + 3/6 workaround → **kết luận C** → wrap `RegionsEditorControl` thay build mới `KzMotionRegionsEditor`.

Bước 3 — Wrap UserControl + 3-workaround template

W1 — Background image (khi Kz control không có `BackgroundImage/Source`):

Dùng `Grid` với 2 layer chồng, Kz control với `Background="Transparent"`:

```
<UserControl x:Class="..." xmlns:kz="using:KztekComponentAvalonia.Controls">
    <Grid>
        <!-- Layer 0: Background image -->
        <Image Source="{Binding BackgroundImage}"
            Stretch="Uniform"
            HorizontalAlignment="Stretch"
            VerticalAlignment="Stretch" />
        <KzZoneEditor />
    </Grid>
</UserControl>
```

```
VerticalAlignment="Stretch"/>
<!-- Layer 1: Kz component transparent để lộ image dưới -->
<kz:KzZoneEditor x:Name="PART_ZoneEditor"
    Background="Transparent"
    IsHitTestVisible="{Binding IsDrawEnabled}"
    ZoneAdded="OnZoneAdded"/>

</Grid>
</UserControl>
```

W2 — Mode toggle (khi Kz control không có DrawingMode / IsEditable):

Bind `IsHitTestVisible` vào VM property:

```
[ObservableProperty] private bool _isDrawEnabled;
// AXAML: IsHitTestVisible="{Binding IsDrawEnabled}"
// Khi false → user click không có tác dụng. Tùy ý bind Cursor riêng cho visual feedback.
```

Bonus: sau mỗi lần user vẽ xong 1 item, reset `IsDrawEnabled = false` trong `ZoneAdded` handler để parity WinForms OnMouseUp behavior (`drawingMode = DrawingMode.None` sau khi drag xong).

W3 — Coordinate scale (khi Kz control dùng canvas coords, cần image pixel coords cho caller):

Chuyển scale logic từ code-behind Form gốc → ViewModel/wrapper UserControl:

```
public KzRect[] MotionRectangles // image pixel coords cho caller
{
    get
    {
        var zones = PART_ZoneEditor.Zones; // canvas coords
        var imgSize = _vm.BgImageSize; // image pixel size
        var canvas = PART_ZoneEditor.Bounds;
        if (imgSize.Width == 0 || canvas.Width == 0) return Array.Empty<KzRect>();

        float sx = (float)imgSize.Width / (float)canvas.Width;
        float sy = (float)imgSize.Height / (float)canvas.Height;
        return zones.Select(z => new KzRect(
            (int)(z.Bounds.X * sx), (int)(z.Bounds.Y * sy),
            (int)(z.Bounds.Width * sx), (int)(z.Bounds.Height * sy)
        )).ToArray();
    }
    set
    {
        PART_ZoneEditor.ClearZones();
        if (value == null) return;
        var imgSize = _vm.BgImageSize;
        var canvas = PART_ZoneEditor.Bounds;
        if (imgSize.Width == 0 || canvas.Width == 0) return;
        float sx = (float)canvas.Width / (float)imgSize.Width;
        float sy = (float)canvas.Height / (float)imgSize.Height;
        foreach (var r in value)
        {
            var zone = new KzZone(
                new Rect(r.X * sx, r.Y * sy, r.Width * sx, r.Height * sy),
                string.Empty, "#00FF00");
            PART_ZoneEditor.AddZone(zone);
        }
    }
}
```

QUAN TRỌNG: Set `MotionRectangles` (setter) **PHẢI** defer đến `Loaded` event của caller Form, không phải constructor — vì `PART_ZoneEditor.Bounds = 0` trước khi layout xong → scale ra 0/0 → không zone nào add được.

```
public frmRegions()  
{  
    InitializeComponent();  
    this.Loaded += (_, _) => // DEFER - không set trong constructor  
    {  
        if (_initialRects != null) regionsEditor.MotionRectangles = _initialRects;  
    };  
}
```

Áp dụng lại (How to reuse)

Khi gặp task migrate WinForms control → Avalonia trong KZTEK:

1. **Không quyết định vội** — trước khi chọn "build mới control", luôn probe Kz component tương đương bằng `MetadataLoadContext`.
2. Liệt kê 5-6 câu hỏi tính năng CỤ THỂ (không phải "có tương đương không") → match với API probe → phân loại A/B/C.
3. **Ưu tiên kết luận C** khi có thể — tiết kiệm 80% effort, giữ Kz brand nhất quán.
4. Template AXAML 3-workaround W1/W2/W3 tái dùng nguyên xi cho các control tương tự (crop image, annotation editor, drawing surface).
5. **BẮT BUỘC defer set collection property đến `Loaded` event** — `Bounds = 0` trong constructor.
6. Ghi verification report `docs/architecture/<slug>/<task>-<KzControl>-verification.md` với đầy đủ API list + 6 câu hỏi + kết luận A/B/C — làm audit trail để reviewer G5.99 verify.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ `MetadataLoadContext` LOAD FAIL nếu thiếu bất kỳ Avalonia dep DLL nào trong resolver → thêm `avalonia.controls.datagrid, avalonia.themes.fluent, avalonia.desktop, avalonia.skia` (theo `.deps.json` của Kz assembly).
- ⚠ XML doc `KztekComponentAvalonia.xml` chỉ ghi tag types + 1-2 method, KHÔNG có full property/event → dựa vào XML sẽ đánh giá SAI, phải dùng Reflection.
- ⚠ `IsHitTestVisible=false` KHÔNG đổi `Cursor` — user vẫn thấy cross-cursor nếu Kz component set. Cần bind `Cursor` riêng nếu muốn visual feedback đầy đủ.
- ⚠ Kz component collection thường `ReadOnlyList<T>` (không có setter) — dùng `AddZone/RemoveZone/ClearZones` methods để build up từ code programmatic; setter không tồn tại là intentional (immutability from consumer).
- ⚠ Scale coord logic — DO NOT dùng float32 khi canvas hoặc image quá nhỏ (< 100px); risk truncation lỗi 1-2px ở rìa. Với đa số camera resolution (640×480+) an toàn.
- ⚠ Wrap pattern KHÔNG hoạt động cho control cần override render pipeline sâu (VD: multi-frame video player) — trường hợp đó phải build mới (kết luận B).

Tham chiếu

- Project: Kztek.Camera migrate WinForms → Avalonia (branch `feat/avalonia`).

- Task: G3.6 [KzZoneEditor](#) API verification + G5.1 [RegionsEditorControl](#) wrap implementation.
- Verification report: [docs/architecture/kztek-cameras-to-avalonia/G3.6-KzZoneEditor-verification.md](#).
- Code example:
[1.Source/Kztek.Cameras.Avalonia/SettingPage/RegionsEditorControl.axaml.cs](#).
- Related lessons: [avalonia-migration-from-winform.md](#), [avalonia-migration-review-checklist.md](#), [avalonia-property-subscription-before-visual-tree.md](#).